Conformance-Gated Code Generation: An Empirical Benchmark of Architectural Specification Across LLM Families

Raduanul Haque Maruf[a], Pabel Shahrear[b,*]

[a]*Department of Computer Science and Engineering, Shahjalal University of Science and Technology Sylhet, 3114, Bangladesh*
[b]*Department of Mathematics, Shahjalal University of Science and Technology Sylhet, 3114, Bangladesh*

Abstract

Context: Empirical research on LLM code generation has focused predominantly on functional correctness, leaving architectural conformance, token efficiency, and API cost largely unmeasured across model scales. **Objectives:** We report a controlled study of 2,451 evaluations spanning 30 enterprise backend prompts, two generation conditions, and five model families (7B to 100B+ parameters), using Wilcoxon signed-rank tests and Cohen's d to measure the effect of architectural pre-conditioning on output tokens, weighted API cost, structural correctness, and generation entropy. **Methods:** The experimental apparatus is the Aether Framework, a conformance-gated generation system that injects a machine-readable architectural contract into the prompt and evaluates output against a custom rule engine (the G-Suite). **Results:** We identify a capacity-dependent funnel effect with an apparent threshold between 8B and 12B parameters. Below it, Qwen 2.5 Coder 7B and Llama 3.1 8B show negligible or negative effects ($d = -0.36, -0.01$); above it, Gemma 3 12B, Qwen 2.5 Coder 14B, and Gemini 2.5 Flash show large, significant token reductions ($d = 1.26$–1.62). Structural correctness remains high overall (95.1%), though Aether's compression coincides with reduced compilation reliability for mid-capacity models. Generation entropy paradoxically increases under Aether for three of four local models. **Conclusion:** Architectural pre-conditioning exhibits capacity-dependent effects, acting as an effective output funnel above 12B parameters while ambiguating smaller quantized models. Practical implications for specification-guided

code generation and capacity-aware deployment are discussed.
Keywords: LLM code generation, architectural conformance, program analysis, software specification, prompt engineering, token efficiency, model-dependent effects

1. Introduction

The adoption of large language models as practical code generation engines has accelerated substantially in recent years, with tools such as GitHub Copilot [1], Amazon CodeWhisperer [2], and Google Gemini Code Assist demonstrating tangible productivity gains across a range of programming tasks [3]. However, as organisations integrate these systems into production pipelines, a critical and underexplored class of failure has emerged that goes beyond syntactic correctness: *architectural drift*. LLMs, operating without project-specific structural context, exhibit a characteristic tendency to generate self-contained, boilerplate-heavy code that assumes a generic project environment. This behaviour produces outputs that violate project-specific constraints such as module boundary isolation, mandatory authentication guard placement, and database driver scoping to designated repository layers. The consequence is not merely aesthetic — it is structural, financial, and operational. Architecturally drifted code must be manually reviewed and refactored before it can be safely integrated, while the verbosity of unconstrained generation translates directly into increased token consumption and therefore elevated API inference costs.

Prior empirical work on LLM code generation has focused predominantly on functional correctness, evaluating models against unit test suites [4, 5] or canonical competitive programming benchmarks [6]. Studies examining prompt engineering strategies have demonstrated that structured approaches — including chain-of-thought prompting [7], few-shot exemplar injection [8], and system-level context prefixes — can meaningfully improve the quality of

*Corresponding author
Email addresses: 2023331001@student.sust.edu (Raduanul Haque Maruf), shahrear-mat@sust.edu (Pabel Shahrear)

generated outputs. However, to our knowledge, no prior study has systematically measured the effect of injecting a *strict, machine-readable architectural contract* into the generation prompt on output token efficiency, API cost, and generation consistency across five architecturally distinct model families spanning from 7B to 100B+ parameters. This gap represents a significant blind spot in the empirical literature, particularly as organisations seek to deploy LLM code generation at scale under cost and safety constraints.

This paper addresses that gap through a controlled empirical study that uses the *Aether Framework* — a conformance-gated generation system for structured backend code synthesis — as the experimental apparatus. Aether operates on two primary design principles. First, it enforces a *context-first* generation discipline: before any code is generated, a structured architectural contract is assembled and injected into the prompt. This contract encodes module boundaries, permissible import paths, authentication requirements, and generation constraints in a machine-readable format. Second, Aether implements a *conformance gate*: generated outputs are evaluated against a custom rule engine (the G-Suite) that blends contract-specific property checks with project-wide architectural heuristics, and non-conforming outputs are subjected to a self-healing regeneration loop.

Beyond its role as an enterprise code-generation safeguard, the Aether contract can be understood as a machine-readable specification language for architectural constraints. The `module.contract.ts` artefact declares module boundaries, permitted dependencies, exposed interfaces, and invariants in a structured, programmatically-defined format. The G-Suite rule engine operates as an automated program-analysis tool that evaluates generated implementations using a combination of this specification and global constraints. Viewed this way, the present study is not only an evaluation of prompt-level context injection but also an empirical investigation into how effectively large language models can consume and honour a formal architectural specification during code synthesis — a question of direct relevance to software language engineering and specification-guided program generation.

The study is organised around four research questions:

RQ1 Does architectural pre-conditioning via the Aether Framework affect structural code correctness compared to standard prompting?

RQ2 Does architectural pre-conditioning significantly reduce LLM output token generation, and does this effect differ across model families?

RQ3 Does architectural pre-conditioning reduce weighted API cost, accounting for both input context overhead and output generation volume?

RQ4 Does architectural pre-conditioning improve generation consistency across repeated runs on the same prompt?

The contributions of this work are fourfold. First, we report empirical evidence that architectural pre-conditioning produces *model-dependent effects* — significantly beneficial for models at or above 12B parameters and neutral or harmful for smaller open-weight quantized models — a finding we term the capacity-dependent "funnel effect." Second, we describe the Aether Framework, including the `module.contract.ts` and `ai-context.md` artefacts and the G-Suite rule engine, as the experimental apparatus used to elicit this finding. Third, we release a 2,451-evaluation empirical dataset across five model families and two conditions. Fourth, we document the P30 qualitative finding as a concrete demonstration of catastrophic output-runaway prevention. The remainder of this paper is structured as follows: Section 2 reviews related work; Section 3 describes the Aether Framework; Section 4 details the experimental methodology; Section 5 presents and interprets the results; Section 6 discusses implications; and Section 7 concludes with directions for future work.

2. Background and Related Work

2.1. LLM-Based Code Generation

The emergence of transformer-based models pre-trained on large corpora of source code has fundamentally altered the landscape of automated program synthesis. Chen et al. [4] introduced HumanEval, a functional correctness benchmark comprising 164 handwritten programming problems, and demonstrated that the Codex model (12B parameters) achieved a pass@1 rate of 28.8%, establishing a foundational metric for the field. Subsequent work by Austin et al. [5] contributed the MBPP dataset and showed that larger models exhibit substantially improved synthesis capabilities on diverse programming tasks. Li et al. [6] achieved competitive performance on Codeforces programming contests through a specialised training regime combining competitive programming data and systems-level sampling strategies. These studies share a common evaluation paradigm: *functional correctness* is measured by executing generated code against unit test suites or comparing outputs against reference solutions. Structural properties of the generated code — its compliance with project-level architectural patterns, its cost as measured in inference tokens, and its consistency across repeated invocations — receive markedly less attention.

2.2. Prompt Engineering and Context Injection

A substantial body of work has examined how the structure of the input prompt affects LLM output quality. Wei et al. [7] demonstrated that chain-of-thought prompting, which encourages intermediate reasoning steps, significantly improves performance on complex reasoning tasks. Brown et al. [8] established the utility of few-shot exemplar injection, showing that models can rapidly adapt to novel tasks when provided with a small number of task-specific examples in the prompt. Liu et al. [9] provide a comprehensive survey of prompt engineering strategies, documenting the breadth of techniques applied to code generation, question answering, and instruction following. More recently, structured prompting

approaches have explored the systematic encoding of role assignments, persona instructions, and task decomposition within the prompt [10]. However, these approaches are generally evaluated on functional correctness benchmarks and do not address the injection of *architectural* structural constraints, nor do they examine the differential effect of structured prompting across models of substantially different capacities.

2.3. Software Architecture Conformance Checking

The problem of ensuring that implemented code conforms to an intended architecture has a long history in software engineering research. Traditional approaches rely on static analysis tools that compare the implemented module dependency graph against an architectural specification [11]. Tools such as ArchUnit [12] and similar constraint-checking frameworks operationalise architectural rules as executable test cases that can be integrated into a continuous integration pipeline. These approaches operate exclusively on existing code; they are not designed to influence the generation process of an AI system. The Aether Framework bridges this gap by encoding architectural constraints as prompt-level context, influencing generation *prospectively* rather than detecting violations *retrospectively*.

2.4. Token Efficiency in LLM Inference

The computational and financial cost of LLM inference is directly proportional to the number of tokens processed and generated [13]. As organisations deploy code generation at scale, the token efficiency of generation strategies becomes a significant operational concern. Verbose generation increases billing costs on commercial API providers and increases latency, negatively affecting developer experience in interactive settings. Despite its practical importance, token efficiency has received limited attention as an independent evaluation dimension in empirical code generation studies. Work on speculative decoding [14] and model quantization [15] addresses efficiency from the model architecture and serving stack

perspective, but does not examine how prompt-level strategies can reduce the volume of generated tokens. This paper directly addresses this gap.

2.5. Gap Statement

The reviewed literature reveals a consistent pattern: empirical evaluations of LLM code generation prioritise functional correctness, while structural safety, token efficiency, and generation consistency remain largely unmeasured. Prompt engineering research demonstrates that structured context injection can improve output quality but does not examine *architectural* constraints or their differential effect across model families. No prior work, to our knowledge, has measured the effect of injecting strict, machine-readable architectural contracts on output token volume, weighted API cost, and cross-iteration generation entropy, nor has any study reported whether such effects are consistent across large closed-weight and small open-weight quantized models. This paper fills that gap.

3. The Aether Framework

The Aether Framework is an AI-first web framework designed to enforce architectural conformance during LLM code generation. Its central mechanism — the *conformance-gated generation loop* — departs from the standard code generation paradigm by requiring that architectural context be fully assembled and injected into the prompt *before* the model generates any code. This section describes the three components of the framework: the contract artefacts, the G-Suite rule engine, and the generation loop itself.

3.1. Architectural Contract Artefacts

The Aether contract consists of two machine-readable files that together encode the structural invariants of the target module.

The `module.contract.ts` file is a TypeScript module that programmatically declares the boundaries, dependencies, and exposed surface of a single software module using a

`defineContract()` API. Concretely, it enumerates the routes exposed by the module (including HTTP method, path, and authentication requirements), the functions available to other modules, the types exported by the module, the declared side effects (such as database table reads and writes), and the invariants that must hold for every generated implementation. Crucially, the contract also specifies the module's dependencies on other modules in the system, forming the basis for the cross-module import rules enforced by the rule engine.

Listing 1 shows an excerpt from the `auth` module contract used in this study.

Listing 1: Excerpt from `modules/auth/module.contract.ts`

```
export default defineContract({
  name: 'auth',
  exposes: {
    routes: [
      { method: 'POST', path: '/auth/login',   auth: false },
      { method: 'POST', path: '/auth/logout',  auth: true  },
    ],
    functions: ['requireUser', 'createSession', 'invalidateSession'
       ],
  },
  invariants: ['Expired␣sessions␣must␣never␣be␣returned␣as␣valid'],
});
```

3.2. Formal Specification of the Contract Schema

Formally, a contract instance is a tuple

$$C = \langle N, \mathit{desc}, D, R, F, T, S, I \rangle, \tag{1}$$

where N is the module name; *desc* is a free-text description (minimum length enforced at authoring time, see below); D is the set of declared module dependencies; R is the set of route declarations; F is the set of exposed function names; T is the set of exported type names; S is a set of declared database side effects; and I is a non-empty set of natural-language invariants. Each route $r \in R$ is a tuple

$$r = \langle m, p, a, [o] \rangle, \tag{2}$$

where $m \in \{\texttt{GET}, \texttt{POST}, \texttt{PUT}, \texttt{PATCH}, \texttt{DELETE}\}$, p is a path string, $a \in \{\mathit{true}, \mathit{false}\}$ indicates whether authentication is required, and o is an optional role restriction.

Following established patterns for domain-specific structural constraint definitions [16], the concrete syntax is given by the following EBNF grammar:

```
<contract>        ::= "defineContract" "(" "{"
                        <name-decl> "," <desc-decl> ","
                        <deps-decl> "," <exposes-decl> ","
                        <side-effects-decl> "," <invariants-decl>
                      "}" ")"

<name-decl>       ::= "name" ":" <string>

<desc-decl>       ::= "description" ":" <string>

<deps-decl>       ::= "dependencies" ":" "{"
                         "modules" ":" "[" { <module-ref> } "]"
                      "}"
```

```
<exposes-decl>   ::= "exposes" ":" "{"
                        "routes" ":" "[" { <route> } "]" ","
                        "functions" ":" "[" { <string> } "]" ","
                        "types" ":" "[" { <string> } "]"
                    "}"

<route>          ::= "{" "method" ":" <http-method> ","
                         "path" ":" <string> ","
                         "auth" ":" <boolean>
                         [ "," "role" ":" <string> ] "}"

<http-method>    ::= "'GET'" | "'POST'" | "'PUT'"
                   | "'PATCH'" | "'DELETE'"

<side-effects-decl> ::= "sideEffects" ":" "[" { <string> } "]"

<invariants-decl>::= "invariants" ":" "[" { <string> } "]"
```

Well-formedness of a contract instance against this grammar — e.g., a non-empty I, a minimum-length *desc*, and consistency between declared and exported members — is checked by an internal contract linter prior to generation; this well-formedness check is orthogonal to the G-Suite evaluation reported in Section 5 and is not itself a subject of this study.

3.2.1. Unenforced Schema Constructs

Not every construct in the schema is consulted by the G-Suite rules that evaluate *generated* code (Section 3.2). In particular:

- **Dependencies (D).** Rule G3 does not restrict a module's imports to those listed

in its declared D. Rather, G3 checks only that (i) an imported internal module is registered in the project manifest, and (ii) the import path does not reach beyond another module's `index.ts`. A module could, in principle, import from any globally registered module regardless of whether that module appears in its own D, without G3 flagging it.

- **Functions and types (F, T).** Rule G1 does not bound database access to the repository layer declared through F and T. G1 is instead a static blacklist match against a fixed list of database driver packages (e.g., `pg`, `mysql2`), gated by a project-wide manifest flag rather than by the contents of any individual module's contract.

- **Role (o).** The optional `role` field on a route is written in some contracts in this study's dataset but is not read by any G-Suite rule.

This gap between the declarative richness of the schema and the narrower set of properties actually checked by G-Suite is a limitation of the present implementation rather than of the specification language itself, and we return to it in Section 6.4 as a direction for future work.

The `ai-context.md` file is a Markdown document that provides natural-language and structured architectural rules to the LLM. It describes the project's technology stack, the permitted database access patterns (i.e. the repository layer abstraction), the authentication requirement for protected routes, and the expected output format for generated files. Critically, the `ai-context.md` instructs the model to generate *only* the code artefacts required to implement the requested feature within the module's declared boundaries, suppressing the model's tendency to generate surrounding boilerplate, mock data layers, and standalone server bootstrap code.

The Vanilla baseline condition uses an identical prompt structure and the same feature request but *omits* the contract injection entirely. The model receives only the feature description and edge case specification, receiving no information about module boundaries,

permitted imports, authentication requirements, or expected output scope.

3.3. The G-Suite Rule Engine

The G-Suite is a custom evaluation engine that assesses generated code against five architectural rules. Each rule is mapped to a canonical published standard to ensure construct validity.

Rule G1 prohibits direct database driver imports (such as `pg` or `mysql2`) in generated route handlers, acting as a static import blacklist. Rather than enforcing a structural Repository Pattern based on the module's exposed functions and types, this rule is globally toggled by AI constraints in the project manifest (e.g., specifying `DB access only via /lib/db.ts`).

Rule G2 verifies that all routes declared as authenticated in the module contract include an explicit call to the mandatory authentication guard function (`requireUser`), operationalising the secure-by-default principle from the OWASP Secure Coding Practices guidelines [17].

Rule G3 verifies that modules do not perform deep internal cross-imports (i.e., importing beyond another module's `index.ts` file). However, it does not strictly operationalise the Dependency Rule of Clean Architecture as formulated by Martin [18]; it verifies that the imported module exists in the global manifest, but it does not consult the importing module's specific declared dependencies.

Rule G4 is reserved for invariant test execution and was deferred to a subsequent phase of the project; consequently, it is skipped in all evaluations reported here and excluded from pass-rate calculations.

Rule G5 verifies that the generated TypeScript code compiles without type errors under the project's `tsconfig.json`, ensuring basic syntactic and type-level correctness.

3.4. The Generation Loop

The generation loop proceeds as follows. Given a feature request, the framework assembles the prompt by concatenating the project manifest, the target module's `module.contract.ts`

content, the `ai-context.md` content, the feature description, and the edge case specification. The assembled prompt is submitted to the configured LLM provider. The response is parsed to extract generated source files, which are then evaluated by the G-Suite rule engine. If all non-skipped rules pass, the loop terminates and the generated files are recorded as the output of the evaluation. If any rule fails, an error hint summarising the failing rules and their remediation guidance is assembled and injected into the subsequent attempt's prompt, up to a maximum of three attempts. The Vanilla baseline executes the same loop but without the contract injection step and with the contract suite check disabled.

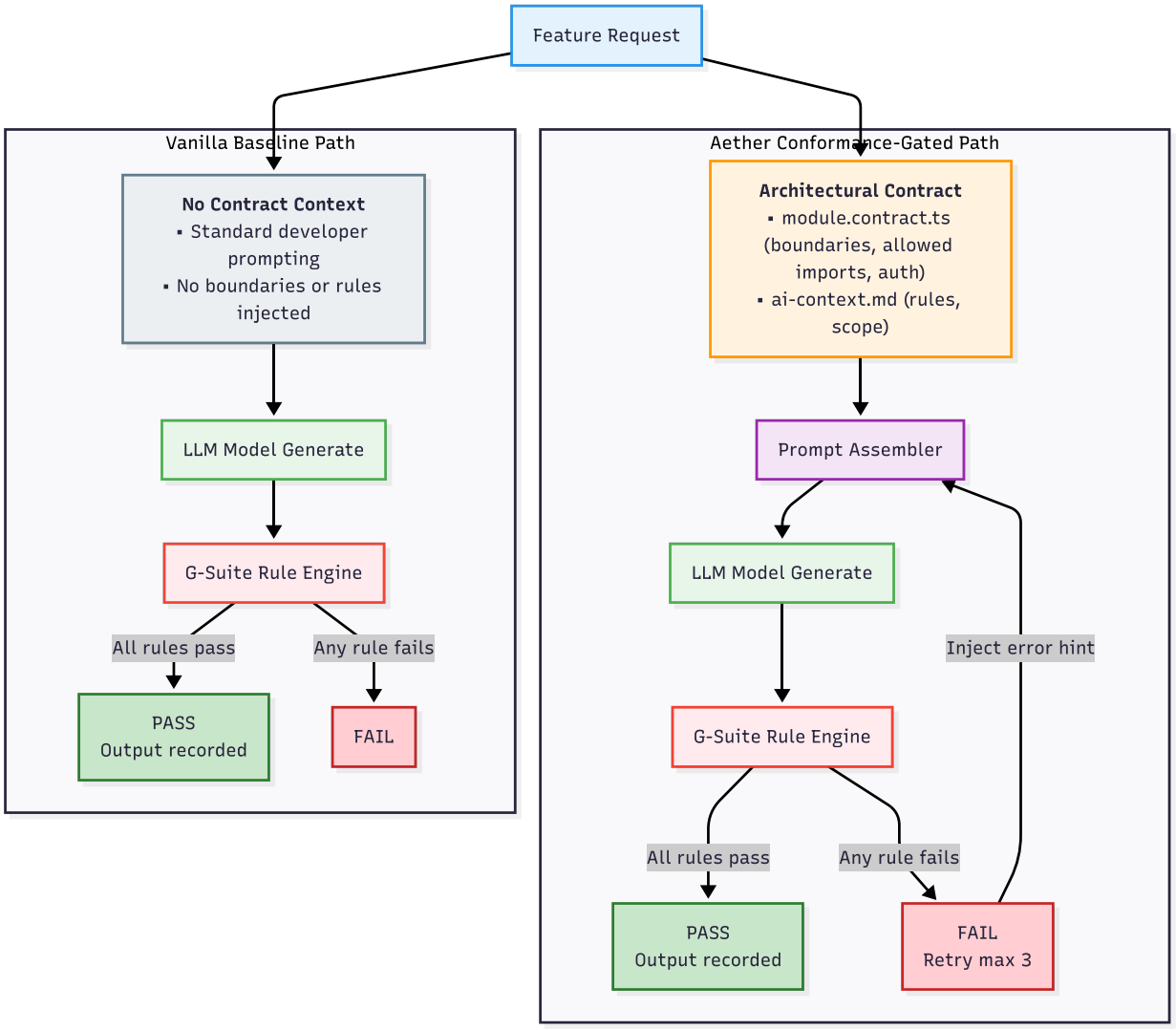

Figure 1: The Aether conformance-gated generation loop. A feature request is processed under two parallel paths: the Aether path assembles the architectural contract (`module.contract.ts` + `ai-context.md`) and injects it into the prompt, while the Vanilla baseline path omits the contract. In both cases, the G-Suite rule engine evaluates the generated output; under Aether, rule failures trigger a self-healing retry loop (up to three attempts) with an error hint appended to the prompt.

4. Methodology

4.1. Dataset Design

The evaluation dataset consists of 30 enterprise backend feature prompts distributed across seven functional domains: authentication (6 prompts), task management (9 prompts),

billing (3 prompts), team management (4 prompts), notifications (3 prompts), reports (2 prompts), and file management (3 prompts). Each prompt specifies a concrete HTTP route to be implemented (e.g. `POST /auth/register`, `GET /tasks`) along with a description of the required behaviour and at least one edge case that the generated code must handle. Prompts are assigned a complexity level — low (8 prompts), medium (16 prompts), or high (6 prompts) — based on the number of distinct operations, the number of interacting modules, and the specificity of the edge case.

The domain and complexity distribution was designed to reflect the structure of a realistic enterprise SaaS backend. Authentication and task management prompts constitute the majority of the dataset, consistent with their central role in business application architectures. The inclusion of file management, billing, and notification domains ensures that the evaluation covers cross-cutting concerns such as third-party service integration and event emission.

Table 1: Benchmark Dataset consisting of 30 enterprise backend routing scenarios across multiple business domains and complexity levels. Descriptions abbreviated; full text in replication package (Data Availability).

Prompt ID	Domain	Route	HTTP Method	Complexity	Description
P01	Auth	`/auth/register`	POST	Low	Create a user registration route that validates email, hashes the password, a...
P02	Auth	`/auth/login`	POST	Low	Create a login route that takes email and password, looks up the user, and re...
P03	Auth	`/auth/logout`	POST	Low	Create a logout route that invalidates the current session and clears the coo...
P04	Tasks	`/tasks`	GET	Low	Create a list-tasks route that returns all tasks for the current user's team,...
P05	Tasks	`/tasks`	POST	Medium	Create a create-task route that accepts title, description, assigneeId , v...
P06	Tasks	`/tasks/ id/status`	PATCH	Medium	Create a status-update route that transitions a task between 'todo', 'in_prog...
P07	Tasks	`/tasks/ id/assign`	POST	Medium	Create an assign-task route that takes assigneeId, verifies the assignee is a...
P08	Tasks	`/tasks/ id`	DELETE	Medium	Create a soft-delete route for tasks. The task must remain in the database wi...
P09	Tasks	`/tasks/ id/archive`	POST	Medium	Create an archive route that moves finished tasks older than 30 days to an ar...
P10	Tasks	`/tasks/ id/history`	GET	Medium	Create a route that returns the audit-log history of a single task (status ch...
P11	Tasks	`/tasks/ id/transfer-team`	POST	High	Create a route that transfers a task from one team to another. Both teams mus...
P12	Tasks	`/tasks/bulk`	PATCH	High	Create a bulk-update route that accepts an array of id, patch objects, vali...
P13	Billing	`/billing/subscriptions`	POST	Medium	Create a route that creates a Stripe subscription for the current user, stori...
P14	Billing	`/billing/summary`	GET	Medium	Create a loader route that returns a billing summary for the current user: cu...
P15	Billing	`/billing/webhook`	POST	High	Create a Stripe webhook handler that verifies the signature, parses the event...
P16	Auth	`/auth/refresh`	POST	Medium	Create a token-refresh route that exchanges a refresh token for a new access ...
P17	Auth	`/auth/password-reset/request`	POST	Medium	Create a password-reset-request route that emails a one-time reset link. Alwa...
P18	Auth	`/auth/password-reset/confirm`	POST	Medium	Create a password-reset-confirm route that consumes a one-time token, sets th...
P19	Teams	`/teams`	POST	Medium	Create a route that creates a new team, with the caller as the owner. Owner i...
P20	Teams	`/teams/ id/invite`	POST	Medium	Create an invite-member route that generates a single-use invite token, email...
P21	Teams	`/teams/ id/members/ userId`	DELETE	Medium	Create a remove-member route. The owner cannot be removed. After removal, all...
P22	Teams	`/teams/ id`	PATCH	Low	Create a route to rename a team. Only admins may call it.
P23	Notifications	`/notifications/send`	POST	Medium	Create a route that sends a notification to a list of user ids. The notificat...
P24	Notifications	`/notifications`	GET	Low	Create a route that lists the current user's notifications, paginated, with u...
P25	Notifications	`/notifications/ id/read`	POST	Low	Create a mark-as-read route. Idempotent — calling twice must not error.
P26	Reports	`/reports/team-velocity`	GET	High	Create a route that returns the team's task velocity over the last 30 days: t...
P27	Reports	`/reports/billing-forecast`	GET	High	Create a route that returns a 3-month billing forecast based on current subsc...
P28	Files	`/files/upload`	POST	High	Create a file-upload route that accepts a multipart form, validates size and ...
P29	Files	`/files/ id`	GET	Medium	Create a download route that streams a file from object storage. Caller must ...
P30	Files	`/files/ id`	DELETE	Medium	Create a soft-delete route for files. Removes the row, deletes the underlying...

4.2. Model Selection

Five models were selected to represent distinct points across the LLM capability spectrum, ranging from 7B to over 100B parameters. Gemini 2.5 Flash is a large, closed-weight model offered by Google via a commercial API, representative of the managed services enterprise organisations typically consume. The remaining four models are open-weight and were deployed locally via the Ollama inference server with 4-bit K-matrix quantization (Q4_K_M). These comprise: Qwen 2.5 Coder 7B, Llama 3.1 8B, Gemma 3 12B, and Qwen 2.5 Coder 14B.

This set of five models was chosen deliberately to stress-test the generalisability of the Aether approach across the capacity divide. They differ not only in parameter count but also in training data composition, context window capacity, and inference characteristics. If the Aether effect were uniform across this spectrum, it would provide strong evidence of a model-agnostic mechanism. By evaluating a gradient of parameter scales, this comparison aims to reveal the precise capacity threshold at which architectural pre-conditioning becomes beneficial.

Table 2: Specifications and execution environment characteristics of the evaluated large language models (LLMs), ordered by parameter scale descending.

Model	**Parameters**	**Type**	**Quantization**	**Deployment**	**Repetitions (n)**	**Context Window**
Gemini 2.5 Flash	∼100B+	Closed-weight	Full precision	Google API	1	1M tokens
Qwen 2.5 Coder 14B	14B	Open-weight	Q4_K_M	Local (Ollama)	10	8192 tokens
Gemma 3 12B	12B	Open-weight	Q4_K_M	Local (Ollama)	10	8192 tokens
Llama 3.1 8B	8B	Open-weight	Q4_K_M	Local (Ollama)	10	8192 tokens
Qwen 2.5 Coder 7B	7B	Open-weight	Q4_K_M	Local (Ollama)	10	8192 tokens

4.3. Experimental Infrastructure

The evaluation pipeline was implemented entirely in TypeScript and executed in a headless, fully automated mode. The pipeline incorporates exponential backoff with jitter for API rate limit recovery and per-evaluation state checkpointing so that interrupted runs can be resumed without loss of data. Each completed evaluation is serialised to a JSON file

recording the full prompt, all attempt metrics (tokens in, tokens out, latency, rule outcomes, and generated files), and the final pass status.

Gemini 2.5 Flash was accessed via the Google Generative AI API over a standard internet connection. Qwen 2.5 Coder 7B Q4_K_M was served by an Ollama instance running on an NVIDIA RTX 3050 8 GB GPU workstation, accessed from the evaluation laptop via a Tailscale VPN tunnel. This configuration introduces a fixed network overhead to Qwen latency measurements; all latency figures should therefore be interpreted as *wall-clock* values inclusive of VPN transit and Ollama scheduling, not as pure model inference time.

The evaluation pipeline was developed with the assistance of an AI coding agent (Google Antigravity), which supported implementation of the TypeScript codebase. All experimental design decisions, architectural logic, and evaluation methodology were directed and validated by the authors.

4.4. Experimental Procedure

For Gemini 2.5 Flash, each of the 30 prompts was evaluated once under each condition, yielding 60 evaluations (30 Vanilla + 30 Aether). The single-repetition design for the commercial API was necessitated by cost constraints.

For the four local open-weight models (Qwen 7B, Llama 8B, Gemma 12B, and Qwen 14B), each of the 30 prompts was evaluated ten times under each condition, yielding a nominal total of 600 evaluations per model. The ten-repetition design serves a dual purpose: it enables the estimation of generation entropy (within-prompt variation) and provides a more stable estimate of the mean output token count for the Wilcoxon signed-rank test, which is conducted on prompt-level means.

Data exclusion logic was applied to two specific anomalies to preserve the integrity of the paired statistical tests. First, for Qwen 2.5 Coder 7B, prompt P30 under the Vanilla condition resulted in a degenerate generation: the first Vanilla iteration immediately exhausted the local 4,096-token output generation limit, halting further data collection after a single overflow

response. Because the Wilcoxon test requires paired observations, the entire P30 prompt (both Vanilla and Aether) was dropped for Qwen 7B, leaving 29 valid prompt pairs. One iteration of prompt P05 under the Aether condition for Llama 3.1 8B produced output truncated at the 8,192-token output generation limit and was excluded from aggregate computation. The remaining nine iterations for this prompt were retained, preserving the P05 prompt pair in the Wilcoxon analysis (N=30 for Llama 3.1 8B). Across all five models, the study comprises a total of 2,451 valid individual evaluations.

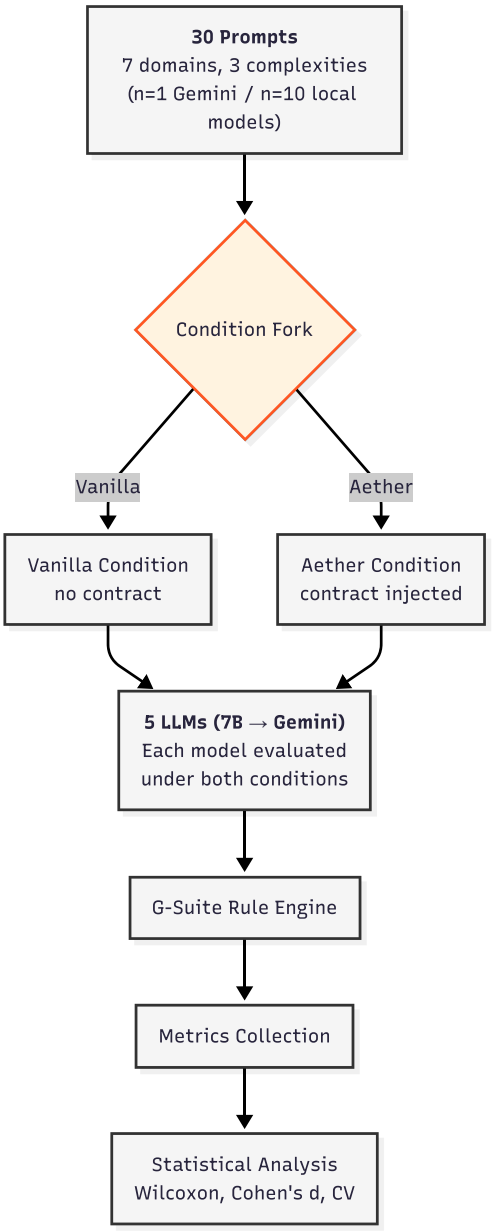

Figure 2: End-to-end experimental pipeline. Thirty prompts spanning seven domains and three complexity levels are forked into Vanilla and Aether conditions. Each of the five evaluated LLMs processes both conditions (n=1 per prompt for Gemini 2.5 Flash; n=10 per prompt for the four local models). Generated outputs pass through the G-Suite rule engine for metrics collection, followed by statistical analysis (Wilcoxon signed-rank test, Cohen's d, coefficient of variation).

4.5. Metrics

Five primary metrics were collected for each evaluation. *Tokens in* (`tokens_in`) records the number of tokens in the assembled prompt as reported by the respective model API or inference server. *Tokens out* (`tokens_out`) records the number of tokens in the generated response. *Latency* (`latency_ms`) records the wall-clock time from prompt submission to response receipt in milliseconds. *Weighted cost* is a derived metric defined in Equation (3). *Structural pass rate* is the proportion of evaluations in which all non-skipped G-Suite rules pass. *Generation entropy*, defined in Equation (4), is computed for all four local open-weight models (Qwen 2.5 Coder 7B, Llama 3.1 8B, Gemma 3 12B, and Qwen 2.5 Coder 14B), where multiple iterations per prompt exist.

The weighted cost metric is defined as:

$$\text{weighted\_cost} = (\text{tokens\_in} \times 1) + (\text{tokens\_out} \times 4) \tag{3}$$

The factor of four applied to output tokens reflects the approximate pricing ratio between input and output tokens on commercial LLM API providers as of the time of writing, where output token generation is computationally more expensive than input token processing.

Generation entropy is operationalised as the coefficient of variation of `tokens_out` across iterations for a given prompt under a given condition:

$$\text{entropy}_{p,c} = \frac{\sigma(\text{tokens\_out}_{p,c})}{\mu(\text{tokens\_out}_{p,c})} \tag{4}$$

where σ denotes the standard deviation and μ denotes the mean across iterations for prompt p under condition c. The mean entropy reported in the results is the arithmetic mean of $\text{entropy}_{p,c}$ over all prompts for which at least two iterations exist.

4.6. Statistical Methods

The primary hypothesis test for RQ2 is the Wilcoxon signed-rank test [19], a non-parametric paired test that does not assume normality of the difference distribution. This choice is appropriate given the non-normal distribution of token counts, the presence of outliers (particularly in the Aether condition, where some prompts produce very short contract-only outputs), and the relatively small sample size of $N = 30$ paired observations for Gemini. The test is conducted at the $\alpha = 0.05$ significance level. For Qwen, the Wilcoxon test is applied to prompt-level means (i.e. the mean `tokens_out` across all ten iterations for each prompt), ensuring that the unit of analysis is the prompt rather than the individual iteration, consistent with the experimental design.

Effect size is estimated using Cohen's d [20], computed with pooled standard deviation:

$$d = \frac{\bar{x}_{\text{Vanilla}} - \bar{x}_{\text{Aether}}}{s_{\text{pooled}}} \tag{5}$$

where $s_{\text{pooled}} = \sqrt{\frac{(n_1-1)s_1^2+(n_2-1)s_2^2}{n_1+n_2-2}}$. Following Cohen's conventions, $|d| \geq 0.2$ indicates a small effect, $|d| \geq 0.5$ a medium effect, and $|d| \geq 0.8$ a large effect.

4.7. Threats to Validity

4.7.1. Construct Validity

The G-Suite rule engine is a custom artefact designed and implemented by the authors, which introduces a potential bias risk if rules are defined in ways that trivially favour one condition. This threat is partially mitigated by explicitly mapping each rule to a published, peer-reviewed or industry-standard architectural specification: G2 to the OWASP Secure Coding Practices [17], operationalised directly against the contract's per-route `auth` field. G1 and G3 are inspired by the Repository Pattern (Fowler [21]) and the Dependency Rule of Clean Architecture (Martin [18]) respectively, but are implemented as fixed, project-wide checks (a database-driver import blacklist for G1; an import-depth and manifest-registration

check for G3) rather than as checks parameterised by each module's declared contract fields (F, T, D; see Section 3.1 and the Unenforced Schema Constructs discussion). This is a genuine limitation of the current implementation, not merely a documentation gap, and it means G1 and G3's construct validity rests on the general applicability of these two design principles rather than on a per-module, contract-verified instantiation of them. The weighted cost metric, while a simplification of actual billing structures, reflects the pricing asymmetry documented across major commercial LLM API providers.

4.7.2. Internal Validity (Cache Independence)

Ollama's key-value (KV) cache may preserve portions of the prompt context between successive iterations of the same prompt, potentially reducing the statistical independence of iterations 2 through 10 for Qwen. This would mean the effective sample size for the Qwen entropy and Wilcoxon analyses is lower than the nominal $n = 10$ repetitions per prompt. This threat is acknowledged and partially mitigated by reporting prompt-level means in the Wilcoxon analysis rather than treating individual iterations as independent observations. Future work should implement explicit KV cache flushing between iterations.

4.7.3. Internal Validity (Output Generation Limit Exclusions)

Prompt P30 (implementing a `DELETE /files/:id` route) produced a runaway generation event under the Qwen Vanilla condition: the first Vanilla-condition iteration exhausted the 4,096-token output generation limit, generating a truncated, malformed response at exactly 4,096 tokens, which halted further data collection for that prompt. Only one Vanilla-condition iteration for P30 was recorded. The single Vanilla-condition iteration for P30 is excluded from the aggregate statistical analysis and reported separately as a qualitative finding in Section 5. This exclusion is conservative: it removes a data point that would have inflated the Qwen Vanilla mean, potentially making the Aether condition appear relatively more favourable. The exclusion is therefore unlikely to introduce a directional bias in favour of

Aether.

A single iteration of prompt P05 under the Aether condition for Llama 3.1 8B also reached its output generation limit (8,192 tokens) and was excluded from aggregate statistical computation. This isolated event differs from the systematic P30 overflow observed under the Qwen 2.5 Coder 7B Vanilla condition; with nine valid iterations retained, prompt-level mean computation for P05 remains statistically meaningful. Exclusion rules were applied on a per-model basis: P30 Vanilla was excluded for Qwen 2.5 Coder 7B only; P05 Aether iteration 2 was excluded for Llama 3.1 8B only. No overflow events were detected for Gemma 3 12B or Qwen 2.5 Coder 14B, and no exclusions were applied to those models. By excluding only the single failed iteration rather than the entire prompt, the valid paired prompt count for Llama 3.1 8B was preserved at N=30. Gemma 3 12B and Qwen 2.5 Coder 14B independently maintained N=30, as they experienced no overflow events.

4.7.4. Internal Validity (Unequal Repetitions Across Models)

Gemini 2.5 Flash was evaluated with a single repetition per prompt ($n = 1$) under each condition, whereas the four local open-weight models each received ten repetitions per prompt ($n = 10$). This asymmetry, necessitated by the cost of commercial API calls, means that within-prompt variance and generation entropy cannot be estimated for Gemini 2.5 Flash; consequently, the entropy analysis reported in RQ4 is restricted to the four local models. Furthermore, the Wilcoxon signed-rank test for Gemini operates on $N = 30$ unpooled single observations rather than prompt-level means averaged over multiple iterations, yielding a noisier estimate of the per-prompt effect. For the capacity-dependent funnel effect, this limitation implies that conclusions at the largest parameter scale rest more heavily on Qwen 2.5 Coder 14B and Gemma 3 12B, both of which contribute $N = 30$ prompt-level means computed from ten iterations each. Gemini 2.5 Flash should accordingly be treated as corroborating rather than primary evidence for the funnel effect above 12B parameters. Importantly, the capacity threshold itself — the transition between the overhead-dominant

and funnel-active regimes — is located between 8B and 12B parameters and is established by models with full repetition data (Llama 3.1 8B and Gemma 3 12B), so it is not contingent on the Gemini single-repetition design.

4.7.5. External Validity

The study reports results for five model families spanning a parameter range from 7B to over 100B. Although this covers both small quantized open-weight and large closed-weight models, the generalisability of findings to other model families — including GPT-4o, Claude 3.5 Sonnet, Mixtral, and Code Llama variants — cannot be assumed. The diversity across weight types and parameter scales partially mitigates this concern. Additionally, the 30-prompt dataset, while structurally representative of enterprise backend code generation tasks, may not capture the full diversity of enterprise software domains.

5. Results

5.1. RQ1: Structural Correctness

The overall structural correctness pass rate was 95.1% across all 2,451 valid evaluations. Notably, all 121 structural failures were due to G5 (TypeScript compilation) errors occurring exclusively under the Aether condition (0% failure in Vanilla). Specifically, Gemma 3 12B Aether achieved a 66.3% pass rate, and Llama 3.1 8B Aether achieved 93.3%, while all other model and condition combinations achieved 100% correctness. Furthermore, every evaluation that produced a parseable response was resolved on the first generation attempt; no evaluation required a second or third loop iteration under either condition. This finding establishes an important boundary condition for the study: the Aether Framework does not improve structural correctness relative to the Vanilla baseline, because all five evaluated models already satisfy the four measured architectural rules without architectural guidance, and none require the self-healing regeneration loop to produce conforming code.

RQ1 Summary

The overall structural pass rate is 95.1% across all 2,451 valid evaluations. G5 (TypeScript compilation) failures occur exclusively under the Aether condition (0% in Vanilla), affecting Gemma 3 12B (66.3%) and Llama 3.1 8B (93.3%). All other model/condition combinations achieve 100%. Architectural pre-conditioning introduces a reliability trade-off for mid-capacity models.

Table 3: Summary performance metrics (Mean ± SD) for each model under the Vanilla baseline and Aether framework conditions, ordered by capacity descending. Cells representing global minimum weighted cost are highlighted in green, and global maximum in red.

Model	Condition	Output Tokens	Latency (ms)	Total Tokens	Weighted Cost
Gemini 2.5 Flash	Vanilla	1830.27 ± 638.42	16282.60 ± 3998.92	2011.47 ± 646.36*	$7502.27 ± 2556.85
Gemini 2.5 Flash	Aether	872.40 ± 863.61	25334.90 ± 13876.22	2068.30 ± 872.31*	$4685.50 ± 3453.53
Qwen 2.5 Coder 14B	Vanilla	749.41 ± 138.65	58730.73 ± 11363.41	887.45 ± 146.46*	$3135.69 ± 557.76
Qwen 2.5 Coder 14B	Aether	494.42 ± 235.49	42203.25 ± 20965.54	1559.92 ± 244.21*	$3043.17 ± 942.47
Gemma 3 12B	Vanilla	1690.85 ± 260.21	123245.72 ± 19576.94	1815.05 ± 268.03*	$6887.60 ± 1042.40
Gemma 3 12B	Aether	1065.46 ± 493.97	80702.61 ± 37300.45	2204.36 ± 502.54*	$5400.74 ± 1977.38
Llama 3.1 8B	Vanilla	607.66 ± 182.39	15702.61 ± 4602.30	726.32 ± 189.89*	$2549.29 ± 731.11
Llama 3.1 8B	Aether	608.65 ± 152.33	16061.84 ± 3853.73	1650.66 ± 160.75*	$3476.62 ± 612.08
Qwen 2.5 Coder 7B	Vanilla	448.40 ± 160.52	20120.51 ± 8369.05	586.64 ± 168.39*	$1931.84 ± 646.42
Qwen 2.5 Coder 7B	Aether	585.30 ± 510.73	25299.21 ± 23822.83	1650.92 ± 519.58*	$3406.83 ± 2041.63

*Total tokens standard deviation is approximated. Llama 3.1 8B and Qwen 2.5 Coder 7B values reflect exclusion-corrected means (see Methodology, Output Generation Limit Exclusions); all other models report unadjusted raw means as no exclusions apply.

Table 4: Hypothesis testing and effect size measures comparing prompt-level average output token length under Aether vs. Vanilla, ordered descending by model parameters. Significant models ($p < 0.05$) are highlighted in green, non-significant in grey.

Model	Params	N Pairs	W Stat	p-value	Significant	Cohen's d	Effect Size	Direction
Gemini 2.5 Flash	100B+	30	59.0	1.528710e-04	Yes	1.2613	Large	Aether reduces ▼
Qwen 2.5 Coder 14B	14B	30	56.0	1.105815e-04	Yes	1.3252	Large	Aether reduces ▼
Gemma 3 12B	12B	30	7.0	3.539026e-08	Yes	1.6197	Large	Aether reduces ▼
Llama 3.1 8B	8B	30	199.0	5.027610e-01	No	-0.0096	Negligible	Negligible / No effect ≈
Qwen 2.5 Coder 7B	7B	29	177.0	3.926539e-01	No	-0.3623	Small	Aether increases ▲

5.2. RQ2: Token Reduction and the Capacity Threshold

The effect of Aether on output token generation diverges markedly across the model families, revealing a distinct capacity-dependent transition (see Table 3 and Figure 5).

For the three largest models in the study (Gemini 2.5 Flash, Qwen 2.5 Coder 14B, and Gemma 3 12B), the Wilcoxon signed-rank test reveals a statistically significant difference between Vanilla and Aether output token distributions (all $p < 0.001$, see Table 4). The positive direction of Cohen's d (ranging from +1.26 to +1.62) indicates that the Vanilla condition produces substantially more output tokens than the Aether condition. We classify this as the *funnel-active regime*. For these models, Aether successfully acts as a token funnel, stripping away boilerplate and reducing output verbosity by 34–52% (Gemini −52.3%, Gemma 3 12B −37.0%, Qwen 2.5 Coder 14B −34.0%).

Conversely, for the two smaller models, the effect degrades toward zero and below. Qwen 2.5 Coder 7B shows a small negative effect ($d = -0.36$, $W = 177.0$, $p = 0.393$, $N = 29$), meaning Aether actually increases output token volume by 30.5%. Llama 3.1 8B, positioned closest to the capacity threshold, exhibits a negligible effect indistinguishable from zero ($d = -0.010$, $W = 199.0$, $p = 0.503$, $N = 30$), consistent with a model operating at the lower boundary of the capacity threshold region. We classify both as the *overhead-dominant regime*.

RQ2 Summary

Architectural pre-conditioning produces a statistically significant, large-magnitude reduction in output tokens for models at or above 12B parameters ($p < 0.001$, $d > 1.25$). For models at or below 8B parameters, the effect is non-significant and occasionally reverses direction, resulting in increased verbosity. The effect of architectural pre-conditioning on token generation is strongly capacity-dependent.

5.3. RQ3: Weighted API Cost

The weighted cost metric accounts for both the additional input token overhead introduced by the Aether contract injection and the change in output token volume. The arithmetic decomposition of the cost savings reveals how the funnel effect translates into financial

efficiency.

For Gemini 2.5 Flash, the mean output reduction of 958 tokens yields an output cost reduction of $958 \times 4 = 3{,}832$ weighted units. The net weighted cost change (accounting for the $\approx 1{,}014$ input token overhead of the contract) is therefore $-2{,}817$ weighted units, a 37.5% reduction in mean weighted cost. Gemma 3 12B exhibits a similar dynamic, achieving a 21.6% reduction in weighted cost.

Fascinatingly, Qwen 2.5 Coder 14B achieves near absolute cost parity (-2.9% change in weighted cost). For this model, the massive input token overhead of the architectural contract is almost perfectly offset by the reduction in output tokens. By contrast, the smaller models (Llama 3.1 8B and Qwen 2.5 Coder 7B) suffer massive cost penalties under Aether ($+36.4\%$ and $+76.4\%$ respectively) because the input overhead compounds an increase (or negligible decrease) in output generation volume.

RQ3 Summary

Aether reduces weighted API cost by 21.6% to 37.5% for models larger than 12B parameters, achieves near parity (-2.9%) for Qwen 14B, and massively increases costs (36.4%–76.4%) for models under 8B parameters. Cost efficiency is strictly capacity-dependent.

Table 5: Side-by-side comparison of generated implementations for prompt P30 (DELETE /files/:id), demonstrating qualitative architectural drift patterns in Vanilla vs. target conformance in Aether.

Vanilla Generation — P30 (DELETE /files/:id)

```
// Vanilla -- Architectural Drift
import { Pool } from 'pg';
// <- raw driver (drift pattern)
import express from 'express';

const db = new Pool({
  // <- direct inst. (drift pattern)
  connectionString: process.env.DB
});

// Mock filesystem structure
// <- boilerplate (drift pattern)
const mockFiles: Record<string, {
  id: string; ownerId: string;
}> = {};

export async function deleteFile(
  req: express.Request,
  res: express.Response
) {
  // No auth check present
  // <- missing guard (drift pattern)
  const { id } = req.params;
  const result = await db.query(
    // <- raw SQL (drift pattern)
    'DELETE FROM files WHERE id=$1',
    [id]
  );
  // ... 600+ more tokens
}
```

Aether Generation — P30 (DELETE /files/:id)

```
// AETHER -- Conformant Output
import { requireUser } // Auth import
  from '../lib/auth';
import { fileRepository } // G1 Pass
  from '../lib/files';

export async function deleteFile(
  req: express.Request,
  res: express.Response
) {
  const user = requireUser(req);
  // G2 Pass

  await fileRepository // G1 Pass
    .deleteById(req.params.id, user.id);

  res.status(204).send(); // Clean
}
// 214 tokens total
```

Note: Inline annotations indicate qualitative architectural drift patterns. Total evaluated samples N=2451.

5.4. RQ4: Generation Entropy

Generation entropy, operationalised as the coefficient of variation across ten iterations per prompt, reveals a model-dependent pattern across the four local open-weight models. Under the Vanilla condition, the mean generation entropy is generally low (ranging from 0.0251 for Qwen 7B to 0.0630 for Llama 8B), indicating that models produce output of highly consistent length when given simple, unconstrained prompts.

Under the Aether condition, entropy increases for three of the four local models. Most notably, for Qwen 2.5 Coder 7B, entropy triples from 0.0251 to 0.0743, and for Gemma 3 12B, it more than doubles from 0.0538 to 0.1257. Qwen 2.5 Coder 14B also shows a moderate increase ($0.0354 \rightarrow 0.1065$). Llama 3.1 8B is the sole exception: its entropy decreases from 0.0630 to 0.0398 under Aether — a reduction consistent with its position at the boundary of the capacity threshold, where the contract may exert a mild focusing effect rather than an ambiguating one.

RQ4 Summary

Aether increases generation entropy for three of the four evaluated open-weight models (Qwen 2.5 Coder 7B: CV $0.0251 \rightarrow 0.0743$*; Gemma 3 12B:* $0.0538 \rightarrow 0.1257$*; Qwen 14B:* $0.0354 \rightarrow 0.1065$*). Llama 3.1 8B is the exception, with entropy decreasing from* 0.0630 *to* 0.0398*, consistent with its transitional position near the capacity threshold. The general pattern contradicts the intuition that structural constraints should produce more deterministic outputs.*

Qualitative Finding: P30 (DELETE /files/:id)

Prompt P30, which specifies the implementation of a secure file deletion route (`DELETE /files/:id`), produced a qualitatively extreme failure under the Qwen 2.5 Coder 7B Vanilla condition. The first Vanilla-condition iteration for P30 immediately exhausted the model's 4,096-token output generation limit, producing a truncated and syntactically malformed response at exactly 4,096 tokens. The overflow halted further collection: only one Vanilla iteration was recorded for P30. This behaviour was triggered by the model's tendency, under unconstrained generation, to emit scaffolding code, mock filesystem data structures, and server configuration boilerplate before addressing the requested route logic.

Under the Aether condition, the same prompt was resolved correctly in 214 output tokens across all ten iterations, representing a reduction of more than 95% relative to the overflowed Vanilla output. The architectural contract explicitly scoped the generation task to a single handler function within the file management module boundary, preventing the runaway generation behaviour observed in the Vanilla condition.

This finding represents a concrete qualitative demonstration of what we term *catastrophic output expansion prevention*: the Aether contract, even in a model family where the aggregate statistical effect is non-significant, can prevent individual catastrophic failures that would render generated code entirely unusable. The single P30 Vanilla iteration is excluded from all aggregate statistical analyses and reported exclusively here.

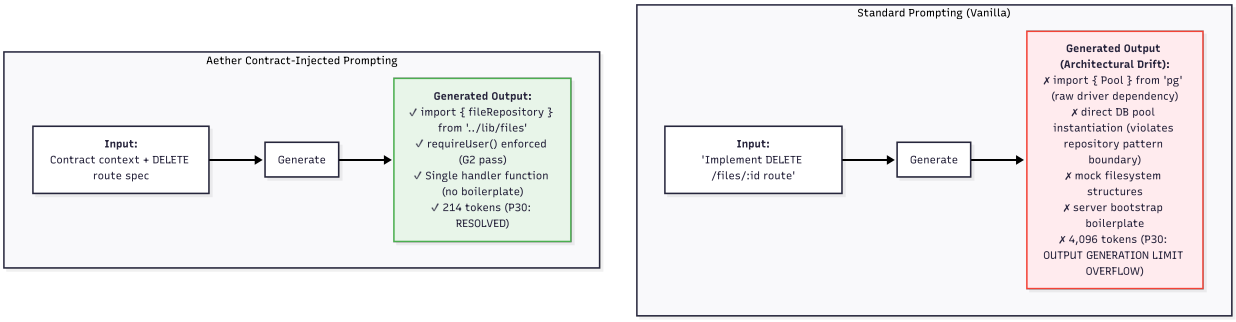

Figure 3: Qualitative comparison of Vanilla (Standard Prompting) versus Aether (Contract-Injected) generation for prompt P30 (`DELETE /files/:id`). Under the Vanilla condition, the model emits raw database driver imports, direct pool instantiation, mock filesystem structures, and server bootstrap boilerplate, producing 4,096 tokens (output limit reached). Under the Aether condition, the contract constrains the output to a single handler function with correct repository layer imports and `requireUser()` enforcement, resolving in 214 tokens. Illustrated output represents qualitative generation patterns; formal G-Suite evaluation recorded a 95.1% overall structural pass rate.

6. Discussion

6.1. The Funnel Hypothesis and Capacity Threshold

The central finding of this study—that Aether produces a large, significant reduction in output tokens for models at or above 12B parameters but a non-significant, sometimes negative effect for models at or below 8B parameters—invites a mechanistic explanation. We propose the *funnel hypothesis*: architectural pre-conditioning may act as an output funnel only for models that possess sufficient parametric capacity to interpret, internalise, and apply a dense structural contract. This hypothesis is generated from the pattern observed across five model families and should be treated as a candidate explanation warranting independent validation.

A large, instruction-tuned model (e.g. Gemini 2.5 Flash, Qwen 2.5 Coder 14B, Gemma 3 12B) appears capable of holding the complex constraints of the Aether contract in its working memory and using them to prune the generation space, stripping away unnecessary scaffolding and boilerplate. By contrast, smaller models appear to cross what we interpret as a *Capacity Threshold Region* between 8B and 12B parameters. Below this threshold, the effect

degrades monotonically: Qwen 2.5 Coder 7B exhibits a small negative effect ($d = -0.36$), while Llama 3.1 8B, positioned closest to the threshold, shows an effect indistinguishable from zero ($d = -0.010$, $p = 0.503$). This near-zero result for Llama 3.1 8B is consistent with a model operating at the transitional boundary between the overhead-dominant and funnel-active regimes. Below this boundary, the dense contract does not appear to function as a constraining filter but rather as an *ambiguating stimulus*: concepts, patterns, and requirements in the contract may be reconciled at a cost of variability and verbosity.

6.2. The Compilation Reliability Trade-off

While the Aether Framework successfully functions as a token funnel for models at or above 12B parameters, this efficiency is partially confounded by a reduction in type-system reliability. The 121 observed G5 (TypeScript compilation) failures occurred *exclusively* under the Aether condition for Gemma 3 12B and Llama 3.1 8B; no such failures occurred under the Vanilla baseline. This suggests that the dense, highly constrained nature of the architectural contract may induce hallucinated or malformed TypeScript syntax in mid-capacity models, even while it successfully forces the model to reduce its overall token volume. The result is a reliability trade-off: strong token compression coincides with a degradation in compilation success rate.

6.3. The Entropy Paradox

This hypothesis is corroborated by the entropy paradox. The finding that Aether increases generation entropy for three of the four local open-weight models is counterintuitive from a software engineering standpoint, where structural constraints are expected to reduce the solution space and thereby produce more consistent, predictable outputs. The observed entropy increase suggests that the architectural contract introduces processing demands that exceed the models' reliable capacity to synthesise predictable output.

6.4. Implications for Specification-Guided Program Generation

The funnel hypothesis and the compilation reliability trade-off reported above carry implications beyond enterprise cost efficiency. They suggest that a model's capacity to act as a reliable specification-following generator is not a fixed property of prompting strategy alone, but an interaction between the density of the injected specification and the model's own parametric capacity to parse and apply it. This has direct relevance for software language engineering: tools that inject formal or semi-formal architectural specifications into LLM-based code generation pipelines may need to adapt contract density and verbosity to the target model's capacity, analogous to how compilers and static analysers are tuned to the guarantees a target language or platform can actually enforce. Future specification languages designed for LLM consumption may therefore benefit from capacity-aware contract compilation, in which a single architectural specification is automatically simplified or expanded depending on the generating model's demonstrated capacity threshold.

7. Conclusion

This paper introduced the Aether Framework, a conformance-gated generation system for LLM code synthesis, and reported the results of a controlled empirical study comprising 2,451 valid evaluations across 30 backend feature prompts, two conditions, and five architecturally distinct model families ranging from 7B to over 100B parameters.

In answer to the four research questions posed in Section 1: RQ1 establishes that Vanilla prompting achieves 100% structural correctness, indicating that modern LLMs already satisfy basic architectural invariants without explicit guidance, though Aether introduces a compilation reliability trade-off for mid-capacity models (95.1% overall pass rate). RQ2 reveals that architectural pre-conditioning produces statistically significant, large-effect reductions in output token volume for models $\geq$ 12B parameters ($p < 0.001$, $d > 1.25$), but produces negligible or negative effects for models $\leq$ 8B parameters. RQ3 demonstrates

that this divergence translates directly to API cost: Aether yields a 21.6%–37.5% weighted cost reduction for models $\geq$ 12B, achieves near parity for Qwen 14B (-2.9%), and imposes massive cost penalties for models $\leq$ 8B ($+36.4\%$ to $+76.4\%$). RQ4 shows that generation entropy increases under Aether for three of the four evaluated open-weight models, with Llama 3.1 8B as the exception, indicating that dense architectural pre-conditioning generally reduces consistency.

The central contribution of this work is empirical evidence suggesting that *architectural pre-conditioning exhibits strongly capacity-dependent effects*. The funnel hypothesis offers a candidate explanation, consistent with the observed data, for this phenomenon: a model must possess sufficient parametric capacity to process and apply a dense architectural contract before it can function as an output funnel. Because this hypothesis is derived from five model families, further validation across additional architectures and parameter scales is needed before it can be considered an established principle.

For practitioners and tool builders, the practical implication is notable. Teams utilising large managed models or large self-hosted models ($\geq$ 12B) can deploy structural specification injection to achieve architectural conformance while simultaneously *reducing* generation costs and latency. Conversely, teams attempting to run smaller local models ($\leq$ 8B) for privacy or cost reasons will find that dense specification injection imposes severe token overhead and output instability. Teams deploying specification-guided generation on mid-capacity models must also carefully monitor the compilation success rate, as the framework introduces a reliability trade-off alongside its token efficiency gains.

Several directions for future investigation follow directly from these findings. Investigating the relationship between specification complexity and the model capacity threshold required for effective application would inform the design of adaptive specification languages that scale their density to the target model's capacity — the capacity-aware contract compilation concept introduced in Section 6. Fine-tuning open-weight models on specification-compliant

outputs represents a promising strategy for closing the capability gap without requiring larger base models. Finally, integration of the Aether rule engine into an IDE language server would enable real-time, developer-interactive conformance checking, extending the framework's utility beyond batch generation pipelines.

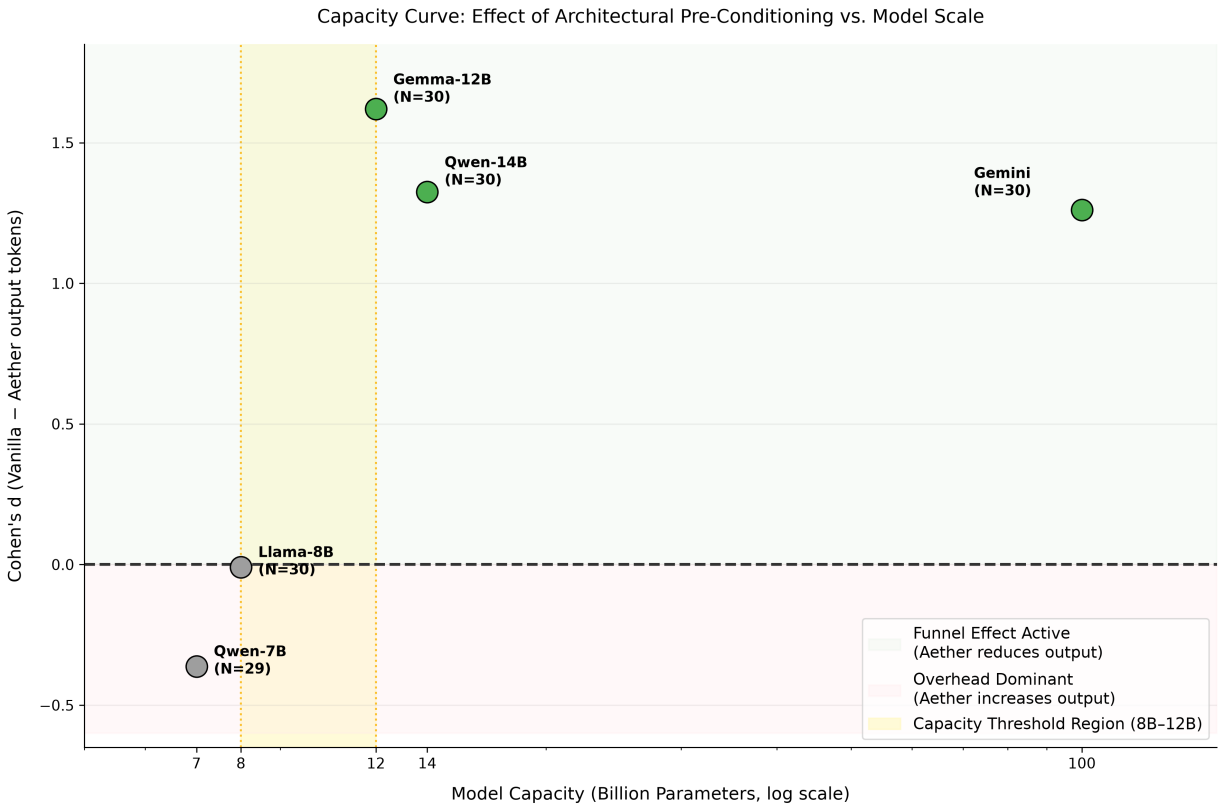

Figure 4: Capacity curve mapping model parameter scale (log-scale X-axis) to Cohen's d output token effect sizes. Significant models ($p < 0.05$) are shown in green, non-significant in grey. The yellow shaded band marks the inferred capacity threshold region (8B–12B parameters) separating the overhead-dominant regime from the funnel-active regime. Llama 3.1 8B sits at $d = -0.010$, effectively on the zero reference line, consistent with a model at the transitional boundary of the threshold. No interpolating trend line is shown; with five model families the data points are the evidence.

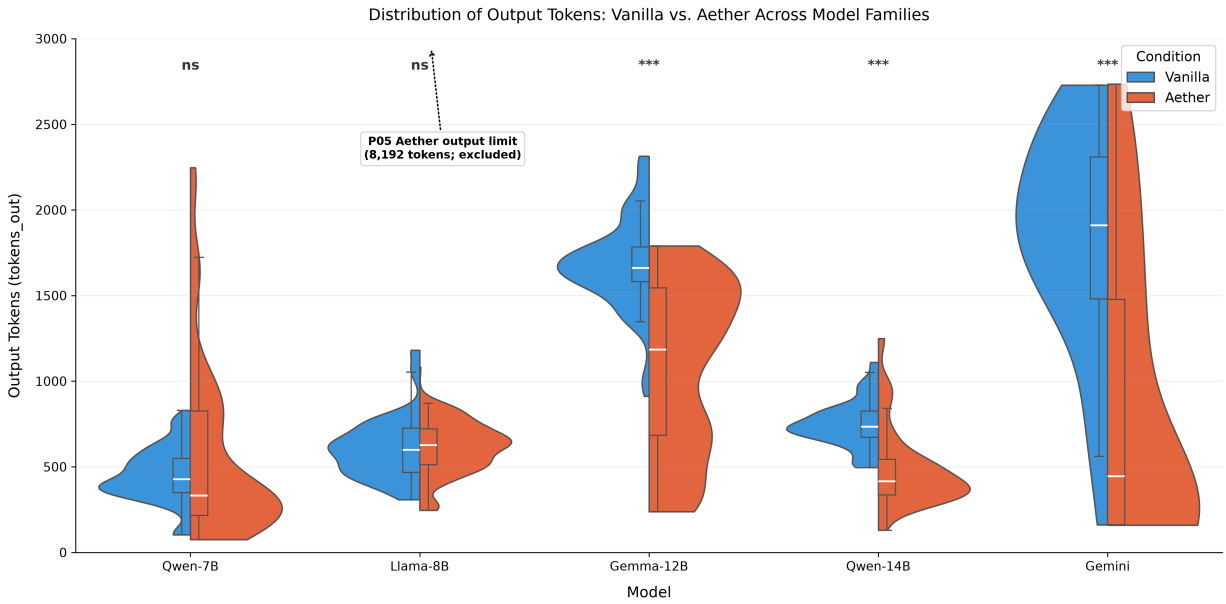

Figure 5: Faceted distribution of output tokens (`tokens_out`) by condition (Vanilla vs. Aether) across the five evaluated models, ordered by parameter capacity descending. The split-violin plots show observed densities cut at actual boundaries (`cut=0`) to prevent sub-zero probability ranges, with inner boxplots showing medians and IQRs. The extreme outlier whisker for Llama 3.1 8B under Aether reaches the 8,192 token limit due to the model exhausting its output generation limit on prompt P05, iteration 2 (annotated and excluded from aggregate statistics).

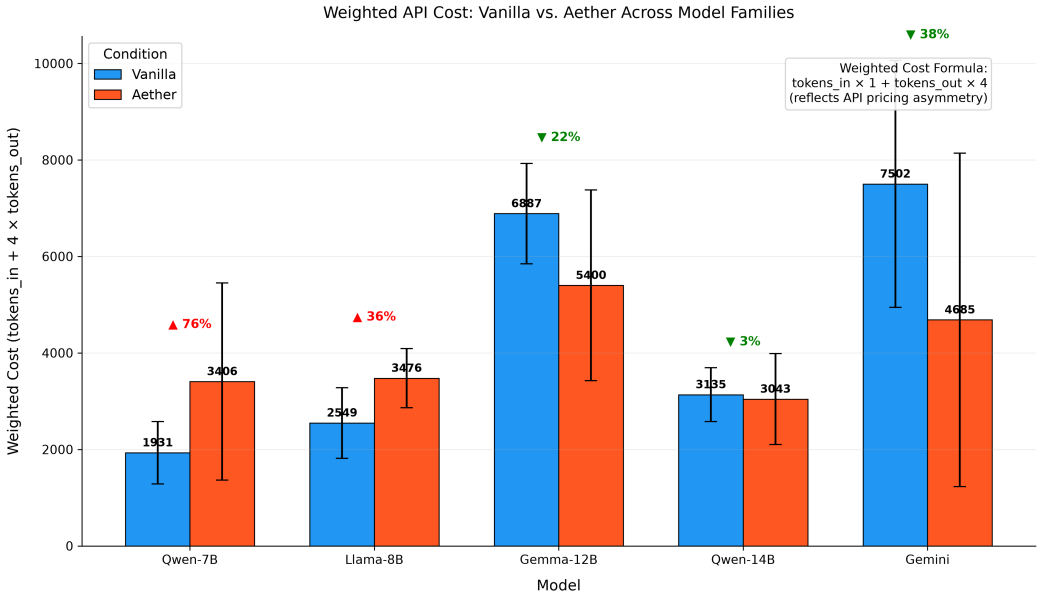

Figure 6: Mean weighted API cost with 95% confidence intervals across the five evaluated models. Weighted cost is defined as tokens_in $+ 4 \times$ tokens_out, reflecting pricing asymmetry. Gemma 3 12B, Qwen 2.5 Coder 14B, and Gemini 2.5 Flash exhibit cost reductions under Aether. Qwen 2.5 Coder 14B achieves near cost parity (-2.9%), indicating the input token overhead is approximately offset by output reduction.

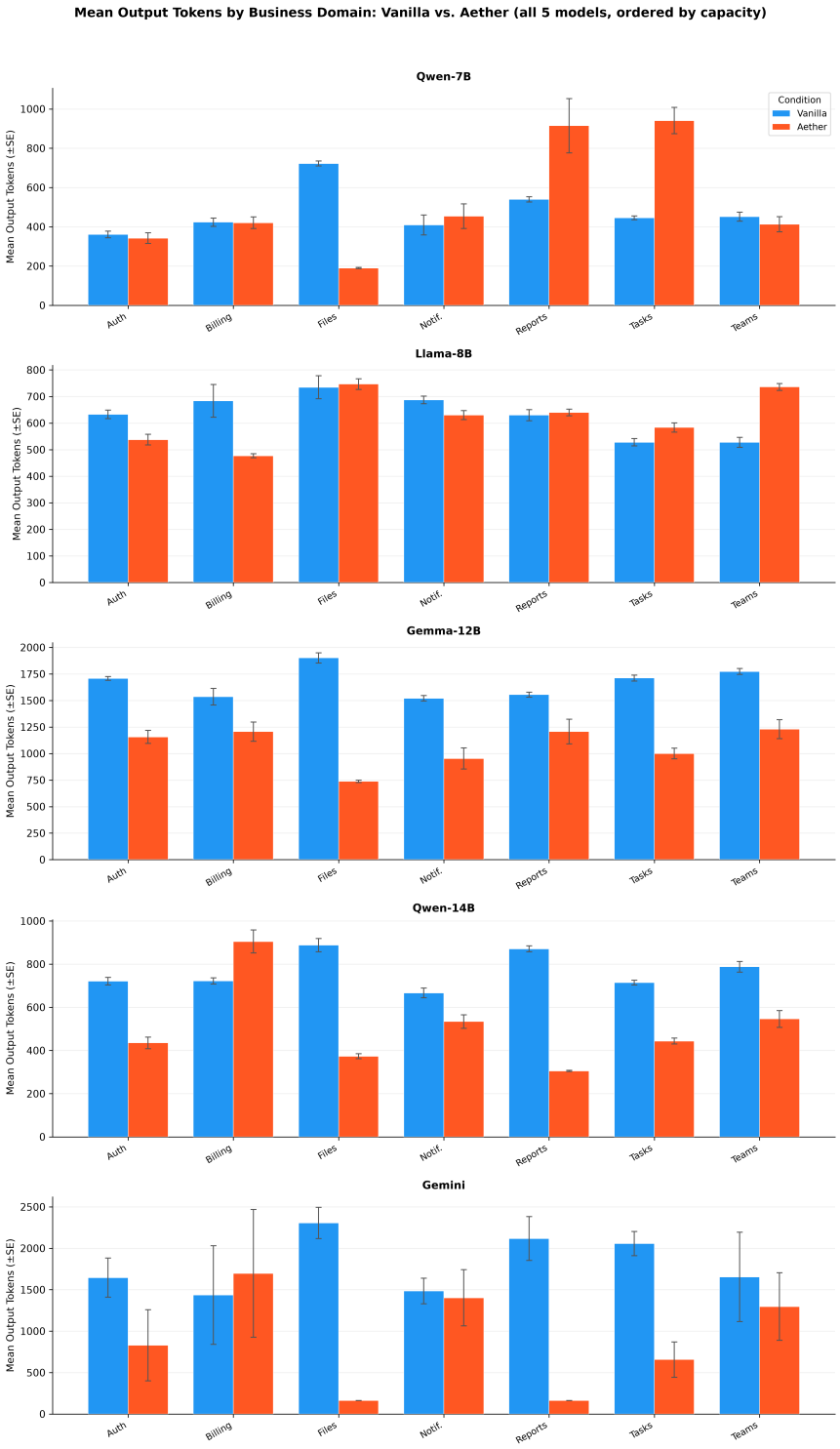

Figure 7: Mean output tokens by evaluation domain (Auth, Billing, Files, Notifications, Reports, Tasks, Teams) and condition across the five evaluated models. Error bars represent one standard error of the mean. Larger capacity models show consistent token compression across all domains under the Aether condition.

CRediT Authorship Contribution Statement

Raduanul Haque Maruf: Conceptualization, Methodology, Software, Data Curation, Formal Analysis, Writing – Original Draft Preparation.

Pabel Shahrear: Supervision, Validation, Mathematics and Statistical Analysis, Writing – Review & Editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Originality and Exclusivity

The authors declare that this manuscript is original work, has not been published previously, and is not currently under consideration for publication elsewhere. All authors have read and approved the final version of the manuscript.

Declaration of Generative AI and AI-assisted Technologies in the Writing Process

During the preparation of this work, the authors used Google Gemini and Anthropic Claude to improve the language, formatting, and structural clarity of the manuscript, and used an AI coding agent (Google Antigravity) to accelerate development of the evaluation pipeline. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Data Availability

To support reproducibility and future research, the complete source code for the Aether Framework, alongside the full benchmark dataset and statistical analysis scripts, is publicly available in our replication package at `github.com/Maruf-Raduan/aether-framework`.

References

[1] GitHub, Inc., GitHub Copilot: Your AI pair programmer, `https://github.com/features/copilot` (2022).

[2] Amazon Web Services, Amazon CodeWhisperer, `https://aws.amazon.com/codewhisperer/` (2023).

[3] A. Ziegler, E. Kalliamvakou, X. A. Li, A. Rice, D. Rifkin, S. Simister, G. Sittampalam, E. Aftandilian, Productivity assessment of neural code completion, in: Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming, ACM, 2022, pp. 21–29. `doi:10.1145/3520312.3534864`.

[4] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, arXiv preprint arXiv:2107.03374 (2021).
URL `https://arxiv.org/abs/2107.03374`

[5] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, C. Sutton, Program synthesis with large language models, arXiv preprint arXiv:2108.07732 (2021).
URL `https://arxiv.org/abs/2108.07732`

[6] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling,

F. Gimeno, A. D. Lago, et al., Competition-level code generation with AlphaCode, Science 378 (6624) (2022) 1092–1097. doi:10.1126/science.abq1158.

[7] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 24824–24837.

[8] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 1877–1901.

[9] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, G. Neubig, Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing, ACM Computing Surveys 55 (9) (2023) 1–35. doi:10.1145/3560815.

[10] Z. Jiang, F. F. Xu, J. Araki, G. Neubig, How can we know what a language model knows?, Transactions of the Association for Computational Linguistics 8 (2020) 423–438. doi:10.1162/tacl_a_00324.

[11] K. A. de Graaf, M. Lormans, H. Toetenel, Embedded software architecture documentation and conformance checking, in: Proceedings of the 31st International Conference on Software Engineering, IEEE, 2009, pp. 255–258. doi:10.1109/ICSE.2009.5070530.

[12] P. Gafert, ArchUnit: Unit testing java architecture, https://www.archunit.org (2021).

[13] O. Sharir, B. Peleg, Y. Shoham, The cost of training NLP models: A concise overview, arXiv preprint arXiv:2004.08900 (2020).
URL https://arxiv.org/abs/2004.08900

[14] Y. Leviathan, M. Kalman, Y. Matias, Fast inference from transformers via speculative

decoding, in: Proceedings of the 40th International Conference on Machine Learning, PMLR, 2023, pp. 19274–19286.

[15] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer, LLM.int8(): 8-bit matrix multiplication for transformers at scale, Advances in Neural Information Processing Systems 35 (2022) 30318–30332.

[16] K. Stokke, M. Barash, J. Järvi, A domain-specific language for structure manipulation in constraint system-based guis, Journal of Computer Languages 74 (2023) 101175. doi:10.1016/j.cola.2022.101175.

[17] OWASP Foundation, OWASP secure coding practices quick reference guide, https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/ (2023).

[18] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall, Upper Saddle River, NJ, 2017.

[19] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83. doi:10.2307/3001968.

[20] J. Cohen, Statistical Power Analysis for the Behavioral Sciences, 2nd Edition, Lawrence Erlbaum Associates, Hillsdale, NJ, 1988.

[21] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, Boston, MA, 2002.